

Deepfake Audio Detection

Performance Report

1. Evaluation Methodology

Accuracy

Percentage of correctly classified samples out of total test samples. Measures overall model correctness across both classes.

Formula:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Code:

```
accuracy = accuracy_score(y_test, y_pred)
```

F1 Score

Harmonic mean of Precision and Recall. Handles class imbalance better than accuracy alone. A score of 1.0 is perfect, 0.0 is worst.

Formulas:

$$\begin{aligned} \text{Precision} &= \text{TP} / (\text{TP} + \text{FP}) \\ \text{Recall} &= \text{TP} / (\text{TP} + \text{FN}) \\ \text{F1 Score} &= 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) \end{aligned}$$

Code:

```
f1 = f1_score(y_test, y_pred)
```

Equal Error Rate (EER)

EER is the point on the ROC curve where the False Acceptance Rate (FAR) equals the False Rejection Rate (FRR). A lower EER indicates better and more balanced detection.

Formulas:

$$\begin{aligned} \text{FAR (FPR)} &= \text{FP} / (\text{FP} + \text{TN}) \text{ [False Acceptance Rate]} \\ \text{FRR (FNR)} &= \text{FN} / (\text{FN} + \text{TP}) \text{ [False Rejection Rate]} \\ \text{EER} &= \text{value where FAR} == \text{FRR} \end{aligned}$$

Approach Used — ROC Curve FPR/FNR Sweep: Instead of using brentq root-finding (which can produce unstable thresholds of 0.0 or 1.0), we used a direct numpy sweep over the ROC curve to find the index where $|\text{FPR} - \text{FNR}|$ is minimized, then averaged FPR and FNR at that point for a stable EER estimate.

Code:

```
fpr, tpr, thresholds = roc_curve(y_test, y_scores)
fnr = 1 - tpr
eer_idx = np.argmin(np.abs(fpr - fnr))
eer = (fpr[eer_idx] + fnr[eer_idx]) / 2
eer_threshold = thresholds[eer_idx]
```

Confusion Matrix

A 2x2 matrix showing the distribution of correct and incorrect predictions for both Fake and Real classes. Reveals per-class performance and error types.

Components:

TP (True Positive) = Fake correctly predicted as Fake

TN (True Negative) = Real correctly predicted as Real

FP (False Positive) = Real incorrectly predicted as Fake

FN (False Negative) = Fake incorrectly predicted as Real

Code:

```
cm = confusion_matrix(y_test, y_pred)
per_class_accuracy = cm.diagonal() / cm.sum(axis=1)
```

2. Results

Metric	Value	Required Threshold	Status
Test Accuracy	88.95%	$\geq 80\%$	PASS
F1 Score	0.8873	≥ 0.80	PASS
Equal Error Rate (EER)	11.05%	$\leq 12\%$	PASS
Fake Class Accuracy	88.86%	$\geq 75\%$	PASS
Real Class Accuracy	89.05%	$\geq 75\%$	PASS

Confusion Matrix:

	Predicted: Fake	Predicted: Real
Actual: Fake	2,106 (TP)	264 (FN)
Actual: Real	248 (FP)	2,016 (TN)

Green = Correct predictions (TP/TN) | Red = Incorrect predictions (FP/FN)

Dataset: The Fake-or-Real (FoR) Dataset — for-norm split | Framework: TensorFlow 2.19 | Model: CNN with Mel Spectrogram features